4th Layer Load Balancer with 6G Network Slicing

Complete System Architecture

1. 6G users / devices

eMBB devices
Video, AR/VR, high-speed data

URLLC devices
Autonomous vehicle, industrial control

mMTC devices
IoT sensors, smart home, massive devices

Traffic characteristics
(from 6G devices)

- IP address
- Port number
- Protocol (TCP/UDP)
- Packet size
- Traffic pattern
- QoS requirement
- Slice type

High bandwidth

Low latency critical

Massive connections

2. 6G network slicing layer

(slice classifier and manager)

Traffic analysis and classification

Inspect packet headers
IP, port, protocol

Analyze traffic features
Latency, bandwidth, packet size

Match with slice rules / ML model

Classify into slice type

eMBB slice
High bandwidth
High throughput
Non-critical

URLLC slice
Ultra-low latency
High reliability
Critical traffic

mMTC slice
Massive conn.
Low bandwidth
Energy efficient

Slice resource management

- Allocate network resources per slice
- Monitor slice usage and QoS
- Dynamic slice adjustment (if needed)

Slice info + packets

3. Layer-4 load balancer

(QoS-aware server selection)

Load balancer core (Layer 4)

Receive incoming packets
(TCP / UDP)

Identify slice type
(from 6G slicer)

Collect real-time server metrics

Calculate weighted score
(slice-specific)

Select best server

Forward to selected server
(NAT / proxy at L4)

Parameters for server selection

- CPU utilization
- Memory utilization
- Network bandwidth
- Latency
- Active connections
- Packet loss

Weighted scoring formula

Score = w_1 x CPU + w_2 x Memory + w_3 x Bandwidth + w_4 x Latency + w_5 x Connections + w_6 x PacketLoss

Slice-specific weights (example)

Parameter	eMBB	URLLC	mMTC
Latency	0.15	0.40	0.20
Bandwidth	0.35	0.15	0.10
CPU	0.20	0.20	0.20
Memory	0.10	0.10	0.15
Connections	0.10	0.10	0.30
Packet loss	0.10	0.05	0.05

4. Backend servers

(application servers)

Server 1 (S1)
IP: 10.0.0.1
CPU: 32%
Memory: 40%
Active conn: 120
Latency: 5 ms

Server 2 (S2)
IP: 10.0.0.2
CPU: 71%
Memory: 60%
Active conn: 400
Latency: 15 ms

Server 3 (S3)
IP: 10.0.0.3
CPU: 44%
Memory: 30%
Active conn: 250
Latency: 7 ms

Server 4 (S4)
IP: 10.0.0.4
CPU: 55%
Memory: 50%
Active conn: 300
Latency: 10 ms

5. Edge and CDN

Edge servers / Content delivery network

- Cache static content
- Reduce latency
- Serve content closer to users
- Handle high bandwidth traffic
- Support global access

Optimized response

End users
Mobile, laptop, web

Traffic statistics, QoS feedback and updated weights

Real-time server metrics

Data / traffic path

Control / feedback path

6. Monitoring and feedback system (control plane)

1. Collect metrics from servers

- CPU, memory, bandwidth
- Latency, active connections
- Packet loss, slice-wise usage

2. Analytics and logging

- Store historical data (database)
- Analyze performance
- Detect overload / failure, reports

3. Dashboard (Web UI)

- Real-time server status
- Slice-wise traffic distribution
- Latency graphs, SLA violation alerts

4. Dynamic adjustment

- Update server weights
- Reassign traffic if overloaded
- Scale resources, maintain slice QoS

1. Parameter rationale - what each metric tells the load balancer

Core score metrics are intentionally kept close to the uploaded architecture, then refined using the repository research. Availability and some SLA limits remain gates rather than score terms to reduce double-counting.

Metric	Why keep / signal	Low value means	High value means	Direction / use	Slice emphasis
Available bandwidth	Capacity left for incoming flow; decisive for high-rate services.	Rate shortfall / congestion risk.	More burst and throughput headroom.	Benefit; x_hat=min(1,B_avail/B_req).	eMBB HIGH; URLLC gate; mMTC LOW
Network latency (RTT)	Fastest direct indicator of deadline / responsiveness risk.	Timely response.	Deadline risk and queueing impact.	Cost; use p95/p99 or conservative RTT.	URLLC HIGH; eMBB MED; mMTC LOW
CPU headroom	Shows compute capacity before queueing grows non-linearly.	Near CPU knee; queue/latency can rise.	Burst capacity.	Benefit; headroom = 1-u.	eMBB MED-HIGH; URLLC gate; mMTC MED
Memory headroom	Protects against cache loss, swapping and OOM-driven failures.	Admission risk / swapping / OOM.	More resident workload capacity.	Benefit; 1-u or free-memory ratio.	mMTC HIGH; eMBB MED; URLLC gate
Queue headroom	Captures backlog that directly turns load into delay.	Backlog and rising delay.	Immediate service capacity.	Benefit; invert normalized queue.	URLLC HIGH; mMTC MED; eMBB LOW
Connection headroom	Measures session/state capacity for many small flows.	Connection-table / session exhaustion.	Massive-device burst capacity.	Benefit; free slots / expected burst.	mMTC VERY HIGH; others LOW
Packet-loss utility	Reflects delivery instability and retransmission cost; strongest for reliability-sensitive traffic.	Unreliable path; retransmission/SLA risk.	Stable delivery.	Benefit after inversion; log-scale loss/failure.	URLLC HIGH; eMBB MED; mMTC LOW
Reliability / SLA success	Outcome metric that catches failures not visible in one resource counter.	Higher failure / SLA violation risk.	Consistent successful service.	Benefit; hard gate + score; log failure prob.	URLLC VERY HIGH; others MED

Control notes: (1) Availability = mandatory gate for all slices. (2) URLLC distance / latency-radius is a pre-filter; then score measured RTT so propagation distance is not counted twice. (3) Composite load is better used as an overload gate than another score term when CPU/memory/queue are already explicit.

Baseline weights for the 8-parameter score

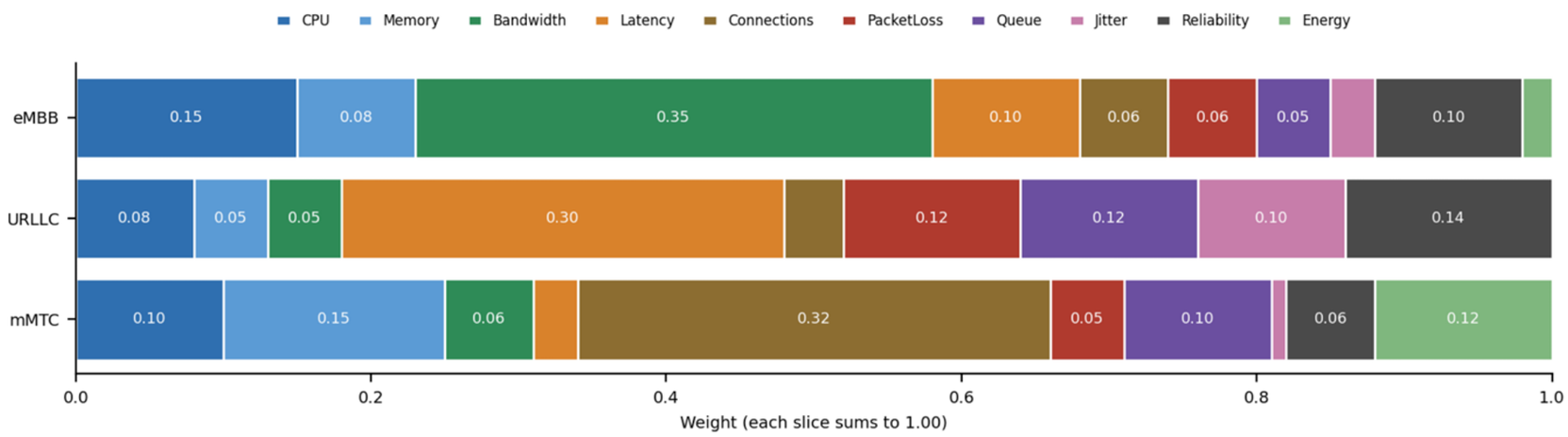
Parameter	eMBB	URLLC	mMTC
Available bandwidth	0.40	0.05	0.05
Network latency / RTT	0.10	0.30	0.02
CPU headroom	0.15	0.08	0.10
Memory headroom	0.10	0.05	0.18
Queue headroom	0.05	0.15	0.12
Connection headroom	0.05	0.03	0.35
Packet-loss utility	0.05	0.14	0.03
Reliability / SLA success	0.10	0.20	0.15
SUM	**1.00**	**1.00**	**1.00**

$$Score = w_{CPU}S_{CPU} + w_{Memory}S_{Memory} + w_{Bandwidth}S_{Bandwidth} + w_{Latency}S_{Latency} + w_{Connections}S_{Connections} + w_{PacketLoss}S_{PacketLoss}$$

$$SelectedServer = \arg\max_i (Score_i)$$

So the behavior is:

```
eMBB  -> bandwidth gets the largest weight
URLLC -> latency gets the largest weight
mMTC  -> active connections gets the largest weight
```



Research evidence / citations

[1] ITU-R M.2160-0 (2023).	Framework and overall objectives for the future development of IMT for 2030 and beyond.
[2] ITU-R M.2083-0 (2015).	IMT-2020 vision; eMBB / URLLC / mMTC service families used as the slice context.
[3] ITU-R M.2410-0 (2017).	Minimum requirements related to technical performance for IMT-2020 radio interface(s).
[4] 3GPP TS 23.501.	System architecture for the 5G System; slice-aware service and network architecture basis.
[5] Bennis, Debbah, Poor (Proc. IEEE, 2018).	URLLC tail / risk / scale rationale: mean-only latency is insufficient for critical traffic.
[6] Hwang & Yoon (Springer, 1981).	Multiple Attribute Decision Making; basis for additive weighted-sum / MCDM scoring.
[7] Cardellini et al. (IEEE Internet Computing, 1999).	Dynamic load balancing with heterogeneous server-state information.
[8] Mitzenmacher (IEEE/ACM Transactions on Networking, 2001).	Power-of-two-choices result; useful implementation note for reducing candidate search cost.
[9] Foukas et al. (IEEE Communications Magazine, 2017).	Network slicing survey; motivates per-slice resource/QoS treatment.
[10] ETSI GS MEC 003.	Multi-access Edge Computing architecture; edge placement / latency context.
[R] github.com/jyotraval/repo.	Repository research artifacts reviewed: gptresearch.md, glm5.3agent.md, glm53chat.md, geminiresearch.txt and duplicate Gemini PDF/DOCX; used as engineering synthesis, not as normative standards.